CAP182 - SS3 Part B

STADIOEquities Account-Activation Model

Project Explained - Following the Data Science Lifecycle

Student: Bernette Mogwera Lesejane - PGDip Data Science, STADIO (SOIT)

October 2026

Code: SS3_PartB.ipynb - Git repository CAP182_Bernette

Platform: Databricks serverless - Data: Unity Catalog volume

This document explains the key aspects of the project in plain language. It walks through the six stages of the data science lifecycle - business understanding, data understanding, data preparation, modelling, evaluation, and deployment - and shows the key code behind each step and what that code does. Numbers quoted throughout come from the verified run of the notebook on 21 October 2026.

Contents

- 1. Executive summary
- 2. Stage 1 - Business understanding
- 3. Stage 2 - Data understanding (loading and auditing 12 client tables)
- 4. Stage 3 - Data preparation (cleaning, leakage control, features, splits)
- 5. Stage 4 - Modelling (SMOTE, Random Forest, XGBoost, ensemble)
- 6. Stage 5 - Evaluation (test metrics, ranking, sensitivity, integrity)
- 7. Stage 6 - Deployment and reproducibility (artifacts, GitHub)
- 8. Glossary of terms
- 9. Conclusion

Executive Summary

The business problem

STADIOEquities is an investment platform. Many people open accounts, but a large share never make their first deposit, which means the account never becomes a revenue-generating client. The business wants to know, as early as possible after sign-up, which accounts are likely to stall so that the support and marketing teams can intervene with the right nudge before the account goes cold.

The solution

A binary classification model was trained on 4,906 real sign-up accounts. For every new account it estimates the probability that the account will activate - that is, make a first deposit within 90 days of registration. The modelling method reuses the approach validated in the SS2 assignment: SMOTE oversampling to balance the training data, a Random Forest and an XGBoost model trained in parallel, and their predicted probabilities averaged into a single ensemble score.

The result (unseen test set, never used during training)

- Accuracy 86.4% - the model is right for about 13 out of every 15 accounts.
- Recall 91.1% for activators - it correctly identifies 9 out of 10 accounts that will deposit.
- ROC-AUC 0.939 and PR-AUC 0.906 - the model ranks accounts very well, which matters more than raw accuracy for building an outreach list.
- 82.9% of true non-activators can be captured by outreach targeted with the model (stall capture).
- Sensitivity checks show the result does not depend on timing-sensitive or leakage-like fields.

All figures, metrics, models, and data artifacts are saved in the repository under results/ and artifacts/, ready to share on GitHub.

Stage 1 - Business Understanding

The question being asked

Which new STADIOEquities accounts are likely to activate (first deposit within 90 days of registration), and which are likely to stall - based only on information available at or shortly after sign-up, before the outcome is known?

Objectives

- Predict an activation probability for every new account.
- Produce a ranked outreach list: flag the accounts most likely to stall for prioritised contact.
- Strictly avoid data leakage - no feature may contain information that only exists because the outcome already happened.
- Deliver a reproducible, well-documented notebook whose artifacts can be shared.

Success criteria

- Good ranking quality on the unseen test set (ROC-AUC, decile lift) - the business uses ranks, not just labels.
- High recall for activators, so genuinely active accounts are recognised.
- Stable performance between validation and test - a sign the model generalises.
- Passing integrity checks: disjoint subsets, no forbidden columns, preprocessor fitted on training rows only.

Ethics and constraints

- Gender is excluded as a predictor - it is a protected attribute.
- KYC/compliance fields are snapshot data taken at extract time; they are tested in a sensitivity analysis rather than trusted blindly.
- The model must remain explainable - feature importances are reported to the client.

Stage 2 - Data Understanding

The dataset

The client supplied 12 CSV tables covering the full customer journey: registration and onboarding, first-session behaviour, demographics, communication and support, pre-deposit interaction, in-app messaging, support tickets, marketing campaigns, KYC/compliance, feedback, device/technical data, and post-deposit activity. The extract is stored in a Unity Catalog volume so the same data path works for everyone who clones the repository.

```
DATA_DIR = Path(os.environ.get("SS3_DATA_DIR",
    "/Volumes/workspace/default/stadioequitiesdataextract"))

def read_table(name):
    return pd.read_csv(DATA_DIR / f"{name}.csv",
                       keep_default_na=False, na_values=[""])
```

What it does: DATA_DIR points at the governed volume copy of the client extract (overridable with an environment variable). keep_default_na=False stops pandas from turning the literal category 'None' (a real answer in investor_experience_level) into a missing value; only truly empty cells become NaN.

Relevance and leakage screening - which tables were kept and why

Table	Verdict	Short reason
01 registration/onboarding	USE	Holds the target; onboarding funnel
02 first-session behaviour	USE	Early engagement before activation
03 demographics/identity	USE	Goal, experience, region (no gender)
04 communication/support	USE	Email engagement and support counts
05 pre-deposit interaction	USE (partial)	Counts kept; timing fields dropped
06 in-app messaging	USE	Push and banner engagement levers
07 support tickets	NOT USED	No timestamp; duplicates table 04
08 marketing campaigns	USE (partial)	Aggregated to one row per account
09 compliance/KYC	USE	KYC status snapshot; tested later
10 feedback/satisfaction	NOT USED	Barrier field only for non-activators
11 device/technical	USE	Sign-up device, OS, VPN, ad-blocker
12 post-deposit activity	NOT USED	Exists only for activated accounts

Tables 07, 10 and 12 are excluded entirely: their content is either known only after the outcome (leakage) or duplicates other tables. The decision log is saved to results/01_table_relevance.csv.

What the audit found

- 14 exact duplicate rows in table 01 were removed: 4,920 raw rows reduce to 4,906 unique accounts - one row per account is required for modelling.

```
t1 = raw["table01_registration_onboarding"].drop_duplicates()
assert t1["account_id"].is_unique
```

What it does: Exact duplicate rows are dropped, and an assert guarantees every account appears exactly once before modelling starts.

- Class balance: 2,813 accounts (57.3%) never activated within 90 days, 2,093 (42.7%) did - a 1.34 : 1 ratio. Mildly imbalanced; handled with SMOTE and class weights.

- Registration dates arrived in four different formats, so raw timestamps were dropped and replaced with day-of-week and hour-of-day features.

- Several category columns had spelling/casing variants of the same value - fixed in preparation (next stage).

- True blank cells are few and structural (e.g. no support ticket raised); they are filled with meaningful labels

rather than dropped.

Stage 3 - Data Preparation

Goal: one clean row per account, with features computed only from information available before activation, and every transformation learned from the training subset only. The raw 73-column account table becomes 169 model-ready features.

Step 1 - Standardise category labels

```
def clean_text(v):
    if isinstance(v, str):
        v = re.sub(r"\s+", " ", v).strip()
        return v if v != "" else np.nan
    return v

def canonicalise(series):
    cleaned = series.map(clean_text)
    key = cleaned.str.lower()
    best = (pd.DataFrame({"k": key, "s": cleaned})
            .groupby("k")["s"]
            .agg(lambda x: x.value_counts().index[0]))
    return key.map(best)
```

What it does: Whitespace is collapsed and trimmed, then each value is matched case-insensitively to the most common spelling of that value. 'Andriod ', 'ANDROID' and 'android' all become one category, which stops the model treating typos as different signals.

Step 2 - Remove leakage and raw identifiers

A leakage column secretly contains the answer. Example: days_to_first_deposit only exists for accounts that deposited, so a model would learn the target from it and then fail in production. Key removals:

Dropped column	Why it leaks / is unsuitable
first_deposit_date, days_to_first_deposit	Only exist if the account deposited
pre_deposit_window_days, total_pre_deposit_minutes	Measured up to the deposit event
conversion_within_30days, campaign_roi_percent	Derived from the conversion outcome
barrier_to_activation (table 10)	Only asked of accounts that did not activate
registration_date (4 formats)	Ambiguous; replaced by day-of-week + hour
gender	Protected attribute - excluded for fairness
account ids, campaign ids	Identifiers carry no signal

```
FORBIDDEN = ["first_deposit_date", "days_to_first_deposit",
             "conversion_within_30days", "campaign_roi_percent",
             "barrier_to_activation", "gender", ...]
leaked = [c for c in FORBIDDEN if c in modelling.columns]
assert not leaked, f"leakage columns present: {leaked}"
```

What it does: A hard assert re-checks at modelling time that none of the forbidden (outcome-derived or protected) columns survived into the modelling table.

Step 3 - Aggregate marketing data to account level

```
mkt_agg = (mkt.groupby("account_id")
       .agg(campaign_exposures=("marketing_campaign_id", "size"),
            campaign_clicks=("user_clicked", "sum"),
            mean_campaign_spend_zar=("campaign_spend_per_user_zar", "mean"),
            n_campaign_channels=("campaign_channel", "nunique"))
       .reset_index())
mkt_agg["campaign_click_rate"] = (mkt_agg["campaign_clicks"]
                                  / mkt_agg["campaign_exposures"])
```

What it does: Table 08 has one row per campaign exposure (8,013 rows for 4,906 accounts). Grouping by account collapses it

into counts, average spend and a click rate - one row per account, which can then be joined without duplicating accounts.

Step 4 - Join tables and fill structural blanks

The nine used tables are joined one-to-one onto the registration table, with an assert that the row count never changes (a safety net against accidental row duplication). Structural blanks get meaningful labels: support_ticket_type 'No ticket', connection_type 'Unknown', unstated demographics 'Not stated'. A missing KYC verification date becomes a real signal: kyc_verification_recorded = 0.

Step 5 - Engineer features

```
modelling["Pre_Deposit_Engagement_Count"] = (
    modelling["search_page_visits"] + modelling["watchlist_items"]
    + modelling["detailed_stock_views"] + modelling["price_alert_set"]
    + modelling["comparison_tool_used"] + modelling["trading_simulation_engaged"]
    + modelling["deposit_page_views"])
modelling["Expected_Push_Opens"] = (
    modelling["push_notifications_received"]
    * modelling["push_notification_open_rate"])
modelling["Early_Session_Share"] = (
    modelling["first_7day_session_count"]
    / (modelling["first_30day_session_count"] + 1))
```

What it does: Four business-meaningful features are added: total pre-deposit engagement, expected push opens (volume x open rate), promo click rate, and how front-loaded a user's early sessions are. These turn raw counts into ratios the client can act on.

Step 6 - Split 70 / 15 / 15, stratified

```
X_trval, X_test, y_trval, y_test = train_test_split(
    X_all, y_all, test_size=0.15, stratify=y_all,
    random_state=RANDOM_STATE)
X_train, X_val, y_train, y_val = train_test_split(
    X_trval, y_trval, test_size=0.15/0.85,
    stratify=y_trval, random_state=RANDOM_STATE)
```

What it does: 3,434 training / 736 validation / 736 test accounts, each with the same 42.7% activation rate (stratify). The validation set monitors learning; the test set is frozen and untouched until final evaluation. Asserts verify no account appears in two subsets.

Step 7 - Cap outliers from training data only

```
caps = {c: float(X_train[c].quantile(0.99)) for c in WINSOR_COLS}
def winsorise(X):
    X = X.copy()
    for c, cap in caps.items():
        X[c] = X[c].clip(upper=cap)
    return X
```

What it does: Extreme values (session seconds, support hours, spend) are capped at the training set's 99th percentile. Caps are computed from training rows only so no information leaks from validation or test into the preparation.

Step 8 - Preprocessing pipeline, fitted on training only

```
preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())]), NUM_COLS),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="constant",
                                 fill_value="Not stated")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
     CAT_COLS)])
preprocessor.fit(X_train_w)   # learns from TRAINING rows only
```

What it does: 54 numeric columns are median-imputed and standardised; 21 categorical columns are filled with 'Not stated' and one-hot encoded into binary columns - producing 169 model-ready features. fit() sees only the training subset; transform() then applies those learned medians, scales and categories to validation and test. The fitted preprocessor is saved (preprocessor.joblib) so future data is prepared identically. This was the key SS2 fix.

Stage 4 - Modelling

The SS2-validated method is retained: SMOTE on the training data, then a Random Forest and an XGBoost classifier whose probabilities are averaged. All fitting uses the training subset only.

Step 1 - Balance the training data with SMOTE

```
smote = SMOTE(k_neighbors=5, random_state=RANDOM_STATE)
X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)
```

What it does: SMOTE synthesises new minority-class examples so the models see a 50/50 training set (3,434 rows grow to 3,938). Critically, SMOTE is applied to the training subset only - validation and test keep the real-world class mix, so reported metrics stay honest.

Step 2 - Random Forest

```
rf_model = RandomForestClassifier(
    n_estimators=100, max_depth=15,
    min_samples_split=10, min_samples_leaf=5,
    max_features="sqrt", class_weight="balanced",
    random_state=RANDOM_STATE, n_jobs=-1)
rf_model.fit(X_train_bal, y_train_bal)
```

What it does: 100 decision trees, each limited to depth 15 and forced to keep at least 5 samples per leaf - the depth and leaf limits prevent any single tree from memorising noise. Class weights add a second layer of imbalance handling on top of SMOTE.

Step 3 - XGBoost, monitored on real (unbalanced) data

```
xgb_model = xgb.XGBClassifier(
    n_estimators=100, learning_rate=0.1, max_depth=6,
    subsample=0.8, colsample_bytree=0.8,
    objective="binary:logistic", eval_metric="logloss",
    random_state=RANDOM_STATE, n_jobs=-1)
xgb_model.fit(X_train_bal, y_train_bal,
    eval_set=[(X_train, y_train), (X_val, y_val)],
    verbose=False)
```

What it does: Gradient boosting builds 100 shallow trees sequentially, each correcting the previous ones' errors. eval_set monitors log-loss on the REAL training and validation subsets while the model learns on the SMOTE-balanced data. The learning curve shows validation loss bottoming at round 40 (0.2645), a mild gap to training loss - the classic overfitting signature that early stopping could act on, and the reason honest evaluation on a frozen test set matters.

Step 4 - Ensemble by averaging probabilities

```
def ensemble_scores(rf, xgb_model, X):
    p_rf = rf.predict_proba(X)[:, 1]
    p_xgb = xgb_model.predict_proba(X)[:, 1]
    return p_rf, p_xgb, (p_rf + p_xgb) / 2
```

What it does: Each model outputs a probability of activation; the ensemble is simply their average. Averaging two differently-biased models smooths individual errors and gives the best trade-off on the unseen test set. An account is labelled 'likely to stall' when the ensemble probability is below the 0.5 decision threshold.

Stage 5 - Evaluation

Performance on the unseen test set (736 accounts, 42.7% activated)

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	Brier
Random Forest	0.859	0.773	0.946	0.851	0.939	0.903	0.095
XGBoost	0.860	0.806	0.885	0.844	0.938	0.909	0.100
Ensemble (SS2)	0.864	0.799	0.911	0.851	0.939	0.906	0.094
Ensemble, validation	0.885	0.836	0.908	0.870	0.953	0.933	0.080

Validation and test scores are close (accuracy 0.885 vs 0.864, ROC-AUC 0.953 vs 0.939), so the model generalises rather than memorising. The ensemble is the best-balanced final model.

Reading the errors in business terms

Quantity	Value	Plain meaning
TN correct non-activators	350	Stallers correctly flagged for outreach
FP missed stallers	72	Predicted active, but never deposited
FN wasted nudges	28	Nudged, but would have deposited anyway
TP correct activators	286	Activators predicted to activate
Stall capture (recall, class 0)	82.9%	Share of stallers the model catches
False positive rate	17.1%	Stallers missed by the flag
False negative rate	8.9%	Outreach spent on accounts that did not need it

At the 0.5 threshold the model catches about 5 out of 6 stalling accounts. Only 72 stallers slip through unflagged, and only 28 nudges are 'wasted' on accounts that would have activated anyway - a cheap error compared with a lost client.

Threshold choice

Validation F1 is nearly flat between thresholds 0.30 and 0.50 (spread 0.008). 0.50 is kept for alignment with SS2; 0.45 is marginally best and is flagged to the client as a tuning option if outreach capacity is limited.

Ranking quality - decile lift

Sorting test accounts by predicted probability and cutting into deciles: the top decile (highest predicted activation) activates far above the 42.7% base rate, while the bottom decile barely activates at all. This is what makes the model operationally useful - the business can contact the lowest-scoring 30% of accounts knowing that list concentrates stallers far better than random targeting.

Sensitivity analysis - is anything leaking by the back door?

Scenario	Features	Accuracy	ROC-AUC	F1
S1 full model	169	0.864	0.939	0.851
S2 no timing-sensitive snapshots	159	0.871	0.941	0.860
S3 also no pre-deposit counts	150	0.864	0.931	0.854

Removing timing-sensitive fields (first-7/30-day session counts, Early_Session_Share) and all KYC snapshot flags barely changes performance. The model is not secretly relying on leakage-like or snapshot features - its signal is genuine onboarding and engagement behaviour.

Integrity checks (all passed)

- Subsets are disjoint - no account appears in two subsets.

- The test subset contains only accounts never used in any fitting step.
- The preprocessor's scaler saw exactly the 3,434 training rows.
- Test ROC-AUC is not suspiciously better than training (overfit guard).
- No forbidden/target column appears among the predictors.

What drives activation (ensemble feature importance)

Feature	Importance	Interpretation
onboarding_completion_status = Completed	0.202	Finishing onboarding dominates
kyc_verification_recorded	0.139	Verified KYC strongly predicts activation
kyc_days_to_verification	0.090	Faster verification, higher activation
onboarding_dropout_step = Completed	0.065	No dropout step helps
time_to_onboarding_completion	0.047	Quick completion helps
kyc_status = Verified	0.043	Compliance state matters
first_30day_session_count	0.018	Early activity helps
Pre_Deposit_Engagement_Count	0.017	Engagement before deposit helps

The story is consistent: accounts that complete onboarding and clear KYC quickly are far more likely to fund their account. That is actionable - the client can streamline onboarding and KYC, and target support at accounts that stall in those steps.

Stage 6 - Deployment and Reproducibility

Artifacts saved for reuse

- Fitted models: artifacts/rf_model.joblib, artifacts/xgb_model.joblib
- Fitted preprocessor: artifacts/preprocessor.joblib (new data is prepared identically)
- Split assignments: artifacts/split_assignments.csv (which account is train/val/test)
- Model-ready splits: artifacts/X_train.csv, X_validation.csv, X_test.csv (+ y_ targets)
- Predictions per split, metrics, threshold tables, sensitivity, decile lift (results/*.csv)
- 11 decision-ready figures in results/figures/ (learning curve, ROC, PR, confusion matrices, calibration, decile lift, feature importance, sensitivity, threshold trade-off)

Environment setup (Databricks)

```
%pip install -q xgboost imbalanced-learn seaborn
```

What it does: The notebook's first cell installs the three libraries not preinstalled on the compute. Everything else (pandas, numpy, scikit-learn, matplotlib, joblib) ships with the runtime, so the notebook runs on any Databricks serverless session.

```
BASE = Path(os.environ.get("SS3_BASE",
    "/Workspace/Repos/mantshetlha@gmail.com/CAP182_Bernette"))
DATA_DIR = Path(os.environ.get("SS3_DATA_DIR",
    "/Volumes/workspace/default/stadioequitiesdataextract"))
```

What it does: Paths are configurable through environment variables, so the same notebook works for any user who clones the repository - only the defaults are workspace-specific.

Sharing via GitHub

- The notebook lives in the Git repository CAP182_Bernette; artifacts/ and results/ are written inside the repo, so a push shares code, outputs and figures together.
- requirements.txt pins the local (VS Code) environment: numpy, pandas, matplotlib, seaborn, scipy, scikit-learn, xgboost, imbalanced-learn, openpyxl, jupyter.
- RANDOM_STATE = 42 is fixed everywhere (splits, SMOTE, both models) - re-running the notebook reproduces every number in this document.

Operational use and monitoring

- Score new sign-ups weekly; contact the lowest-scoring 30% with onboarding support.
- Watch calibration: if predicted probabilities drift from observed activation rates, refit with recent data.
- Retrain periodically (quarterly or on drift); always refit the preprocessor on the new training subset only, using this notebook as the template.

Glossary of Terms

Term	Plain-English meaning
Accuracy	Share of all accounts the model labels correctly
Precision (class 1)	Of accounts predicted to activate, how many really do
Recall (class 1)	Of all real activators, how many the model finds
F1	Balance of precision and recall (harmonic mean)
ROC-AUC	Probability the model ranks a random activator above a random staller
PR-AUC	Ranking quality weighted for the positive (activated) class
Brier score	How close predicted probabilities are to the true outcome (lower better)
Log-loss	Penalises confident wrong predictions (lower better)
SMOTE	Creates synthetic minority examples to balance training data
Data leakage	A feature that contains the answer, unusable in production
Overfitting	Model memorises training noise and fails on new data
One-hot encoding	Each category becomes its own yes/no column
Standardisation	Rescaling numeric columns to mean 0, std 1
Winsorising	Capping extreme outliers at the 99th percentile
Ensemble	Combining models' predictions for a more stable answer
Stratified split	Splitting so every subset keeps the same class mix
Decile lift	Activation rate of a score decile vs the overall average
Confusion matrix	Table of correct and incorrect predictions per class
Random state	Seed that makes 'random' steps reproducible

Conclusion

The project followed the full data science lifecycle on real client data: the business question was translated into a 90-day activation target; 12 tables were audited, screened for leakage and relevance, and reduced to one clean row per account; leakage-safe features were built; the data was split 70/15/15 with every transformation fitted on training rows only; the SS2 SMOTE + Random Forest + XGBoost ensemble was retrained; and the frozen models were evaluated on an untouched test set.

The final model reaches 86.4% accuracy, 0.911 recall and 0.939 ROC-AUC on unseen data, ranks accounts well enough to build a targeted outreach list, and survives sensitivity checks that remove timing-sensitive features. The single clearest insight for the client: accounts that complete onboarding and KYC verification quickly are the ones that fund - so removing friction from those two steps is the highest-value intervention available.

Everything - code, artifacts, metrics and figures - is reproducible from the repository with a single run of SS3_PartB.ipynb on Databricks, and is ready to share on GitHub.